

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность

Образовательная программа: "Информационная безопасность / Information security"

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Защита баз данных»

Выполнил студент:

N3346 / ИББД_ТЗИ_N3 1.4

Суханкулиев Мухаммет / 

ФИО

Подпись

Проверил:

Салихов Максим Русланович / _____

ФИО

Подпись

*Отметка о выполнении (один из вариантов:
отлично, хорошо, удовлетворительно, зачтено)*

Дата

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение	3
1 Мониторинг операций в БД.....	4
1.1 Создание таблицы-лога и триггерной функции	4
1.2 Применение триггеров к таблицам	5
1.3 Демонстрация работы системы логирования	5
2 Шифрование данных	7
2.1 Создание и шифрование таблицы с секретными данными	7
2.2 Демонстрация защиты данных	8
3 Разграничение доступа в БД.....	9
3.1 Создание ролей и назначение привилегий.....	9
3.2 Демонстрация работы системы разграничения доступа	9
Заключение.....	11
Список использованных источников.....	12

ВВЕДЕНИЕ

Цель работы – получение навыков созданию примитивных систем мониторинга, разграничения доступа и шифрования средствами СУБД.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Задачи по мониторингу БД:
 - Создать таблицу-лог, отдельную от основных сущностей БД;
 - Создать для каждой основной таблицы в БД триггер, который срабатывает при любых изменениях в БД. При срабатывании триггер должен вносить в таблицу-лог информацию о том, когда было произведено изменение, со стороны какой роли поступил запрос, какие кортежи поменялись, старые и новые значения;
 - Продемонстрировать работу системы логирования для различных операций и отношений.
2. Задачи по шифрованию данных:
 - Создать таблицу с секретными данными, отдельно от основных сущностей;
 - Зашифровать содержимое данной таблицы, в качестве алгоритма шифрования использовать любой симметричный алгоритм шифрования. Ключ шифрования для данной таблицы не должен храниться в ИС. Ключ шифрования может быть получен из индивидуального пароля для дешифрования суперпользователя. Индивидуальный пароль суперпользователя и ключ шифрования может быть связан через одностороннюю функцию;
 - Продемонстрировать, что даже обладая полными правами администратора, но без знания индивидуального пароля невозможно получить содержимое таблицы с секретными данными;
3. Задачи по разграничению доступа в БД:
 - Создать в СУБД как минимум 2 роли для каждого из классов потребителей информации;
 - С помощью внутренних инструментов СУБД для каждой роли определить набор привилегий по отношению к таблицам БД. Разграничить доступ к представлениям, созданным в 1 ЛР, а также таблицам логирования;
 - Продемонстрировать работу системы разграничения доступа. Зайти за каждую из ролей и показать доступные со стороны каждой роли отношения.

1 МОНИТОРИНГ ОПЕРАЦИЙ В БД

Для отслеживания всех изменений в данных (операций INSERT, UPDATE, DELETE) будет создана система аудита на основе триггеров.

1.1 Создание таблицы-лога и триггерной функции

Первым шагом создадим универсальную таблицу `action_log` для хранения записей аудита. Она будет фиксировать тип операции, время, пользователя, название таблицы и сами измененные данные.

```
CREATE TABLE public.action_log (  
    log_id serial PRIMARY KEY,  
    operation_type char(1) NOT NULL,  
    operation_time timestamp NOT NULL DEFAULT now(),  
    db_user text NOT NULL DEFAULT current_user,  
    table_name text NOT NULL,  
    old_data jsonb,  
    new_data jsonb  
);  
  
COMMENT ON COLUMN public.action_log.operation_type IS 'I - Insert, U - Update,  
D - Delete';
```

Далее создадим триггерную функцию `log_action()`, которая будет вызываться при любом изменении данных и записывать соответствующую информацию в `action_log`.

```
CREATE OR REPLACE FUNCTION log_action() RETURNS TRIGGER AS $$  
BEGIN  
    IF (TG_OP = 'INSERT') THEN  
        INSERT INTO public.action_log (operation_type, table_name, new_data)  
        VALUES ('I', TG_TABLE_NAME, to_jsonb(NEW));  
    ELSIF (TG_OP = 'UPDATE') THEN  
        INSERT INTO public.action_log (operation_type, table_name, old_data,  
        new_data)  
        VALUES ('U', TG_TABLE_NAME, to_jsonb(OLD), to_jsonb(NEW));  
    ELSIF (TG_OP = 'DELETE') THEN  
        INSERT INTO public.action_log (operation_type, table_name, old_data)  
        VALUES ('D', TG_TABLE_NAME, to_jsonb(OLD));  
    END IF;  
    RETURN NULL; -- Результат не важен для AFTER-триггера  
END;  
$$ LANGUAGE plpgsql;
```

1.2 Применение триггеров к таблицам

Теперь применим созданную функцию в качестве триггера для каждой основной таблицы в нашей базе данных: "User", Scooter, Trip, Payment и Maintenance.

```
CREATE TRIGGER user_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public."User"
FOR EACH ROW EXECUTE FUNCTION log_action();

CREATE TRIGGER scooter_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.Scooter
FOR EACH ROW EXECUTE FUNCTION log_action();

CREATE TRIGGER trip_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.Trip
FOR EACH ROW EXECUTE FUNCTION log_action();

CREATE TRIGGER payment_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.Payment
FOR EACH ROW EXECUTE FUNCTION log_action();

CREATE TRIGGER maintenance_log_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.Maintenance
FOR EACH ROW EXECUTE FUNCTION log_action();
```

1.3 Демонстрация работы системы логирования

Продemonстрируем работу созданной системы, выполнив последовательно три разные операции.

```
INSERT INTO public."User" (FIO, PhoneNumber, Email)
VALUES ('Тестов Тест Тестович', '+79001234567', 'test@test.com');

SELECT operation_type, operation_time, db_user, table_name, new_data->>'fio' as
      fio
FROM public.action_log;
```

Data Output Messages Notifications					
					
	operation_type character (1) 🔒	operation_time timestamp without time zone 🔒	db_user text 🔒	table_name text 🔒	fio text 🔒
1	I	2025-09-29 12:16:09.172609	postgres	User	Гбв@й Гбв Гбв@йЕз
2	D	2025-09-29 12:33:22.248565	postgres	User	[null]
3	I	2025-09-29 12:34:10.378005	postgres	User	Тестов Тест Тестович

Рисунок 1 – Логирование операции INSERT в таблицу "User"
(сначала добавилось не в той кодировке)

```
UPDATE public.Scooter SET Status = 'in_use', ChargeLevel = 80 WHERE ScooterID = 1;
```

```
SELECT operation_type, operation_time, table_name, old_data->>'status' as old_status, new_data->>'status' as new_status
FROM public.action_log WHERE operation_type = 'U';
```

Data Output Messages Notifications					
	operation_type character (1)	operation_time timestamp without time zone	table_name text	old_status text	new_status text
1	U	2025-09-29 12:36:05.041405	scooter	available	in_use

Рисунок 2 – Логирование операции UPDATE в таблице Scooter

```
DELETE FROM public.Maintenance WHERE MaintenanceID = 3;
```

```
SELECT operation_type, operation_time, table_name, old_data->>'description' as deleted_description
FROM public.action_log WHERE operation_type = 'D';
```

Data Output Messages Notifications				
	operation_type character (1)	operation_time timestamp without time zone	table_name text	deleted_description text
1	D	2025-09-29 12:33:22.248565	User	[null]
2	D	2025-09-29 12:36:59.752963	maintenance	Плановое ТО

Рисунок 3 – Логирование операции DELETE из таблицы Maintenance (и "User")

Система логирования успешно фиксирует все типы изменений в отслеживаемых таблицах.

2 ШИФРОВАНИЕ ДАННЫХ

Для защиты особо чувствительных данных, таких как токены доступа или API-ключи, используем встроенные криптографические функции PostgreSQL.

2.1 Создание и шифрование таблицы с секретными данными

Сначала необходимо убедиться, что в базе данных активировано расширение pgcrypto.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Создадим таблицу user_api_keys для хранения API-ключей пользователей.

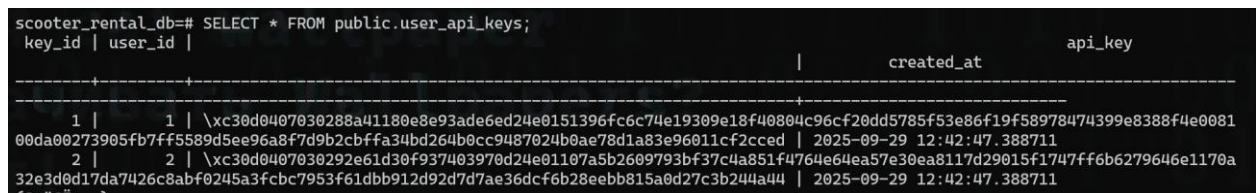
```
CREATE TABLE public.user_api_keys (  
    key_id serial PRIMARY KEY,  
    user_id int NOT NULL REFERENCES public."User" (UserID),  
    api_key bytea NOT NULL,  
    created_at timestamp NOT NULL DEFAULT now()  
);
```

Зашифруем данные при вставке. В качестве пароля для шифрования будем использовать индивидуальный пароль суперпользователя: !super_secret_kickscooter_pwd_2025. Этот пароль передается в функцию шифрования напрямую, что гарантирует, что сам ключ не хранится в базе данных.

```
INSERT INTO public.user_api_keys (user_id, api_key) VALUES  
(1, pgp_sym_encrypt(  
    'scooter_user_1_key_alb2c3d4e5'::text,  
    '!super_secret_kickscooter_pwd_2025'::text  
)),  
(2, pgp_sym_encrypt(  
    'scooter_user_2_key_f6g7h8i9j0'::text,  
    '!super_secret_kickscooter_pwd_2025'::text  
));
```

При просмотре таблицы видны только зашифрованные бинарные данные.

```
SELECT * FROM public.user_api_keys;
```



```
scooter_rental_db=# SELECT * FROM public.user_api_keys;  
key_id | user_id |                created_at                | api_key  
-----+-----+-----+-----  
1 | 1 | 2025-09-29 12:42:47.388711 | \xc30d0407030288a41180e8e93ade6ed24e0151396fc6c74e19309e18f40804c96cf20dd5785f53e86f19f58978474399e8388f4e0081  
00da00273905fb7ff5589d5ee96a8f7d9b2cbffa34bd264b0cc9487024b0ae78d1a83e96011cf2cccd | 2025-09-29 12:42:47.388711  
2 | 2 | 2025-09-29 12:42:47.388711 | \xc30d0407030292e61d30f937403970d24e01107a5b2609793bf37c4a851f4764e64ea57e30ea8117d29015f1747ff6b6279646e1170a  
32e3d0d17da7426c8abf0245a3fbc7953f61dbb912d92d7d7ae36dcf6b28eebb815a0d27c3b244a44 | 2025-09-29 12:42:47.388711
```

Рисунок 4 – Содержимое таблицы user_api_keys в зашифрованном виде

3 РАЗГРАНИЧЕНИЕ ДОСТУПА В БД

Для реализации принципа минимальных привилегий создадим две роли, соответствующие классам потребителей из ЛР1: «Пользователи» (клиенты) и «Технические специалисты».

3.1 Создание ролей и назначение привилегий

Сначала создадим групповые роли без права на вход в систему.

```
CREATE ROLE client_role NOLOGIN;  
  
CREATE ROLE technician_role NOLOGIN;
```

Теперь определим привилегии. Клиенты могут видеть только представления с историей своих поездок и доступными самокатами. Технические специалисты – представления с самокатами для зарядки и журналом обслуживания.

Запрещаем всем по умолчанию доступ к объектам схемы public

```
REVOKE ALL ON SCHEMA public FROM PUBLIC;
```

Разрешаем ролям использовать схему

```
GRANT USAGE ON SCHEMA public TO client_role;  
GRANT USAGE ON SCHEMA public TO technician_role;
```

Выдаем права на представления для клиентов

```
GRANT SELECT ON public.user_trip_history TO client_role;  
GRANT SELECT ON public.available_scooters TO client_role;
```

Выдаем права на представления для тех. специалистов

```
GRANT SELECT ON public.scooters_for_charging TO technician_role;  
GRANT SELECT ON public.maintenance_log TO technician_role;
```

Наконец, создадим конкретных пользователей с правом входа и добавим их в соответствующие группы.

```
CREATE ROLE anna_vasilieva_client WITH LOGIN PASSWORD 'clientpass123';  
GRANT client_role TO anna_vasilieva_client;  
  
CREATE ROLE sergey_mikhailov_tech WITH LOGIN PASSWORD 'techpass456';  
GRANT technician_role TO sergey_mikhailov_tech;
```

3.2 Демонстрация работы системы разграничения доступа

Проверим работу настроенных ограничений, подключаясь от имени созданных пользователей.

```
scooter_rental_db=# SET ROLE anna_vasilieva_client;
SET
scooter_rental_db=> SELECT * FROM public.available_scooters LIMIT 2;
Текущая кодовая страница: 1251
 scooterid | Местоположение | Уровень заряда | Тариф | Стоимость минуты
-----+-----+-----+-----+-----
          2 | Центральный район |          100 | Стандарт |          5.00
          3 | Адмиралтейский район |          45 | Стандарт |          5.00
(2 строки)

scooter_rental_db=> SELECT * FROM public."User" LIMIT 2;
ОШИБКА: нет доступа к таблице User
scooter_rental_db=> SELECT * FROM public.maintenance_log LIMIT 2;
ОШИБКА: нет доступа к представлению maintenance_log
```

Рисунок 7 – Проверка роли anna_vasilieva_client

```
scooter_rental_db=> SET ROLE sergey_mikhailov_tech;
SET
scooter_rental_db=> SELECT * FROM public.scooters_for_charging LIMIT 2;
Текущая кодовая страница: 1251
 scooterid | model | Текущий уровень заряда | Название зоны
-----+-----+-----+-----
(0 строк)

scooter_rental_db=> SELECT * FROM public.Trip LIMIT 2;
ОШИБКА: нет доступа к таблице trip
scooter_rental_db=> SELECT * FROM public.action_log LIMIT 2;
ОШИБКА: нет доступа к таблице action_log
```

Рисунок 8 – Проверка роли sergey_mikhailov_tech

Демонстрация подтверждает, что система разграничения доступа работает корректно, и каждый пользователь имеет доступ только к строго определенному набору данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были успешно решены все поставленные задачи и достигнута основная цель. Были получены практические навыки применения встроенных механизмов СУБД PostgreSQL для обеспечения безопасности данных.

Была реализована комплексная система защиты, включающая:

Систему аудита на основе триггеров, которая позволяет отслеживать все операции с данными, что критически важно для расследования инцидентов и контроля целостности.

Механизм шифрования конфиденциальных данных, который защищает информацию даже в случае несанкционированного доступа к файлам базы данных.

Ролевую модель доступа (RBAC), реализующую принцип минимальных привилегий и надежно изолирующую данные различных категорий пользователей друг от друга.

Разработанные решения повышают общий уровень защищенности информационной системы «Прокат электросамокатов» и являются основой для дальнейшей реализации безопасного сервиса в рамках курсовой работы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы технологий баз данных: учебное пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева; под ред. Е. В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с. – URL: https://drive.google.com/file/d/1TjYbunEjxsbovBiHeYOzBuZrOFonlIRk/view?usp=drive_link
2. Базы данных: Учебник для высших учебных заведений / Под ред. проф. А. Д. Хомоненко. – 6-е изд., доп. - СПб.: КОРОНА-Век, 2009. – 736 с. – URL: https://drive.google.com/file/d/1zIOuO6vdQvb_aVUHGAiucK5MPciUswGf/view?usp=drive_link
3. PostgreSQL 16 Documentation. F.28. pgcrypto. [Электронный ресурс] – URL: <https://www.postgresql.org/docs/current/pgcrypto.html> (дата обращения: 29.09.2025).